DEVELOPER ONBOARDING GUIDE

AI Trading Toolkit

App name is not yet finalised — "EDGE" used in the codebase is a placeholder, subject to change

Standard Developer Procedures

Everything you need to know to start contributing — from first-time setup through daily development workflows, Git branching, pull request reviews, and session handoff procedures.

PROJECT	REPOSITORY	VERSION
AI Trading Toolkit (name TBD)	github.com/ianmaser/trading-toolkit	April 2026

CONTENTS

THE GOLDEN RULES

Never push directly to main.
Always create a branch. Always open a pull request.

1 First Time Setup ONCE ONLY

Follow every step below in order. You only do this once — skip to Section 2 after that.

🖥️ What is a terminal?

The terminal is a text-based window where you type commands to control your computer. Think of it as texting your computer instructions. Everything is case-sensitive.

🍎

Mac

Press **Cmd + Space**, type **Terminal**, press Enter.

🪟

Windows

Press the **Windows key**, type **PowerShell**, press Enter.

📦 Install prerequisites — in this order

1. **Node.js — the engine that runs the app**

   Download from **nodejs.org** (LTS version). Verify: `node -v` → should show `v20.x.x`

2. **Git — the tool that saves and shares code**

   Download from **git-scm.com**. Verify: `git -v` → should show `git version 2.x.x`

3. **VS Code — the code editor**

   Download from **code.visualstudio.com**. This is where you read and review code.

4. **Claude Code — the AI that builds with you**

   In VS Code: Extensions panel → search **Claude Code** → Install. Or run `npm install -g @anthropic-ai/claude-code` in terminal.

🔌 Recommended VS Code extensions

Extension	What it does
ESLint	Highlights code errors as you type
Prettier	Automatically formats code so it stays clean
Tailwind CSS IntelliSense	Autocompletes CSS class names
GitLens	Shows who wrote each line and when

📥 Clone the project

terminal

```
# Download the project
$ git clone https://github.com/ianmaser/trading-toolkit.git
$ cd trading-toolkit
$ npm install
$ code .
```

> ⚠️ **Set up your environment variables**
>
> Ask your partner to share the `.env.local` file. Place it in the root project folder. **Never commit it to Git. Never share it publicly.** It contains API keys for all external services.

✅ Confirm everything works

terminal

```
$ npx tsc --noEmit
# No output = no errors = you're good to go
```

2 Starting Every Session EVERY TIME

Your pre-flight checklist. Takes 2 minutes. Do not skip steps.

1 Open terminal in the project folder

In VS Code: top menu → **Terminal** → **New Terminal**.

2 Get the latest code from main

Always start from an up-to-date main before creating your branch.

terminal

```
$ git checkout main
$ git pull origin main
```

🧯 **If you see "CONFLICT" in the output**

Stop immediately. Two people edited the same file. Message your partner — resolve it together before doing anything else.

3 Install any new dependencies

Safe to run every session — does nothing if nothing changed.

terminal

```
$ npm install
```

4 Confirm the codebase is clean

No output = no errors. If there are errors, fix them or ask your partner before starting work.

terminal

```
$ npx tsc --noEmit
```

5 Create your feature branch

Never work directly on main. Create a branch for what you're building today before touching any code. See Section 5 for naming rules.

terminal

```
$ git checkout -b feature/phase-01-market-data-service
```

6 **Read the session notes**

Open `sessions/` and read the most recent file to find out where the last session left off.

7 **Read the build plan for today**

Open `PLAN.MD` and find your prompt. Read it fully before opening Claude Code.

---

3 Working With Claude Code

Claude Code is an AI that writes code alongside you. It reads your project files and builds features based on your instructions. It has **no memory of previous sessions** — you must give it context every time.

💡 **Starting prompt — use this every time**

"Read CLAUDE.md and PLAN.MD. We're building *[feature name]*. Here's exactly what it needs to do: *[paste the full prompt from PLAN.MD]*"

1️⃣

One prompt at a time

Never ask Claude to build two features in one message — you'll get worse results every time.

🔧

When something breaks

Paste the **full error** + **full file**. Say: "Don't rewrite the whole file. Find the specific problem and fix only that."

🧠

Claude has no memory

Never skip the context prompt. Starting without it is like asking someone to continue a conversation they weren't in.

📝

Session notes via Claude

"Record everything we implemented today in a session note titled *04-16-2026-phase-01.md*"

4 Saving Your Work AFTER EVERY PROMPT

Once a prompt is done and `npx tsc --noEmit` shows no errors, save your work. You are saving to **your branch** — not main.

1 Check what changed

Review every file listed. Make sure you recognise everything before saving.

terminal

```
$ git status
```

2 Stage your changes

`.env.local` is auto-excluded — you will never accidentally commit API keys.

terminal

```
$ git add .
```

3 Commit with a clear message

Format: `feat: Phase {N} Prompt {N} — {short description}`

terminal

```
$ git commit -m "feat: Phase 1 Prompt 5 — market data service"
```

4 Push your branch to GitHub

Push to **your branch**, not main. The first time you push a new branch, Git may suggest adding `--set-upstream` — follow its suggestion.

terminal

```
# Push your branch (not main)
$ git push origin feature/phase-01-market-data-service
```

5 Branching, Pull Requests & Code Review

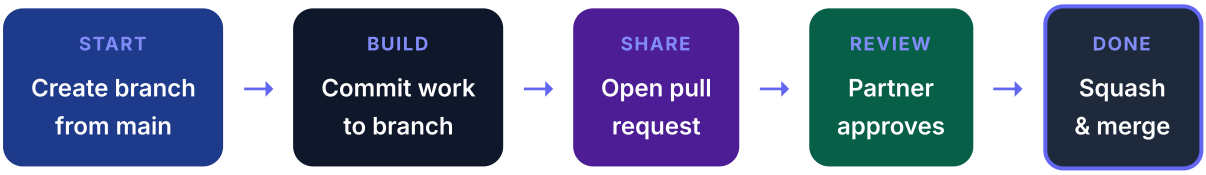
🚫 **Direct pushes to main are not allowed**

`main` is the stable, deployable version of the app. It should always be in a working state. Every change enters main through a branch and pull request — no exceptions. GitHub is configured to reject any direct push to main.

🌿 Branch naming

FEATURE	`feature/phase-01-market-data-service`
FEATURE	`feature/phase-02-watchlist-page`
FIX	`fix/backtest-crash-on-empty-candles`
DOCS	`docs/update-architecture-notes`

🔄 The full workflow — start to finish



📬 Opening a pull request

Once you've pushed your branch and your work is ready for review:

1 **Go to the repository on GitHub**

github.com/ianmaser/trading-toolkit — GitHub will show a banner: **"Compare & pull request"**. Click it.

2 **Fill in the pull request form**

Title uses the same format as commits: `feat: Phase 1 Prompt 5 — market data service`

PR description template

```
## What was built
Brief description of what this PR adds or changes.

## How to test it
Step-by-step instructions so your partner can verify it works.

## Decisions made
Any choices not already in PLAN.MD or ARCHITECTURE.md.

## Notes for reviewer
Anything you're unsure about or want a second opinion on.
```

3 **Assign your partner as reviewer**

Top right of the PR page → **Reviewers** → select your partner. Then click **"Create pull request"**.

🔍 Reviewing a pull request

When your partner assigns you as reviewer:

1 **Read the description and check the files changed**

Click **"Files changed"** tab on the PR to see every added or modified line.

2 **Test it locally (optional but recommended)**

Pull the branch and run the type check to confirm it works on your machine.

terminal

```
$ git fetch origin
$ git checkout feature/phase-01-market-data-service
$ npm install
$ npx tsc --noEmit
```

3 **Leave comments or approve**

Click the `+` next to any line to leave a comment. When you're happy: **"Review changes"** → **"Approve"** → **"Submit review"**.

✅ Merging and cleaning up

1 **Merge after approval**

Click **"Squash and merge"** — this keeps the commit history clean. Then confirm the merge.

2 **Delete the branch**

Click **"Delete branch"** immediately after merging. The branch has served its purpose.

3 **Both partners pull main**

After any merge, both developers should update their local main — don't start a new branch from a stale copy.

terminal — run after every merge

```
$ git checkout main
$ git pull origin main
```

6 Ending Every Session EVERY TIME

🚨 **Claude does not remember your last session**

Every time you open Claude Code, it starts with zero memory of what you built before. Write a session note every time you stop — even if you only worked for 30 minutes and didn't finish anything. Your partner and Claude both depend on it.

1 Ask Claude to write the session note

"Record everything we implemented today in a session note titled `MM-DD-YYYY-short-description.md`"

sessions/MM-DD-YYYY-title.md

```
# Session — MM-DD-YYYY — [Title]

## Who worked on this    [Your name]
## Branch                  [Branch name — is there a PR open?]
## What was built          [Prompt X: description + file locations]
## What is NOT finished    [e.g. Redis caching not wired up yet]
## Left off at             [Exactly what to do next session]
## Decisions made          [Choices not in PLAN.MD or ARCHITECTURE.md]
## Open questions          [Anything the other person needs to decide]
```

2 Commit and push the session note to your branch

Then open a PR if your work is ready for review. If you're mid-feature, just push so your partner can see where you are.

terminal

```
$ git add sessions/
$ git commit -m "docs: session notes MM-DD-YYYY"
$ git push origin your-branch-name
```

---

7 Code Quality Checklist BEFORE EVERY COMMIT

- [ ] `npx tsc --noEmit` — zero errors, no output
- [ ] No `any` types in any new code
- [ ] No hardcoded URLs, API keys, or secrets — everything in `.env.local`

- [ ] TypeScript only — no `.js` files created
- [ ] All new UI components have a loading skeleton and an error state
- [ ] All buttons and interactive elements have an `aria-label`
- [ ] All API inputs validated with Zod
- [ ] Mobile layout works at 375px width

---

8 After Completing a Phase

1 Ask Claude to write tests

"Write tests for everything we just built — happy path, empty states, and error conditions."

2 Manually test the full user journey

Click through everything affected by the phase in your browser. Confirm it works end to end.

3 Update CLAUDE.md

Change the phase status from ⬜ to ✅ in the feature map.

4 Open a pull request and get it reviewed

Don't merge to main until your partner has approved. See Section 5.

9 Weekly Audit EVERY FRIDAY

🔍 **Copy and paste this into Claude Code every Friday**

"Review all the files we've built this week. Look for: any hardcoded values that should be env vars, any missing error handling, any component that's missing a loading state, any Supabase query missing RLS, any TypeScript any types. List everything you find and fix them one by one."

---

10 Git Troubleshooting

🚫 **"Push rejected" when pushing to main**

You should not be pushing to main directly. Create a branch and open a pull request instead — see Section 5.

⚠️ **"My push to my branch was rejected"**

Pull the branch first, then push again: `git pull origin your-branch-name` then `git push origin your-branch-name`

🧯 **"CONFLICT" when pulling**

Two people edited the same file. Stop and message your partner — resolve it together.

🔐 **"I accidentally committed .env.local"**

Message your partner immediately. This is a security issue — handle it together before the commit reaches GitHub.

❓ **"I don't know what state my code is in"**

`git status` — shows changed files. `git branch` — shows which branch you're on. `git log --oneline -10` — shows last 10 commits.

↩️ **"I want to undo my last commit (before pushing)"**

Run `git reset --soft HEAD~1` — un-commits your last save but keeps all changes. Only use before pushing. If already pushed, message your partner first.

---

11 File Map — Where Things Live

What you're looking for	Where it is
What to build next	`PLAN.MD`
How the app is wired together	`ARCHITECTURE.md`
Rules, stack, current phase	`CLAUDE.md`
Database tables and columns	`supabase/migrations/001_initial.sql`
Shared TypeScript types	`types/`
Utility functions (auth, cache, rate limiting)	`lib/`
React hooks (data fetching)	`hooks/`
API services (market data, signals, etc.)	`services/`
Page components	`app/dashboard/`
Reusable UI components	`components/features/` and `components/ui/`
Session handoff notes	`sessions/`
Things flagged to revisit	`TODO/revisit-later.md`
API keys (never in Git)	`.env.local` — shared privately with partner

🤝

REMEMBER

Session notes are your handoff to your partner — and to Claude. Always write one before you log off.